WEEK 6 · DAY 1 · RULE BOOK

Backtesting: Test It on the Past First

Follow it step by step. Every common error and its fix is here.

Your goal today: expose a lucky score, replay the Week 5 forecasts on the past many times, and decide which forecast the NOC should use for next week and for next month's roster. **11 green ✅ ticks** to collect. Time: about 60 minutes. Keep this page open next to Colab.

Before you start (2 minutes)

☐ Open `W6D1-exercise` in Colab → **File** → **Save a copy in Drive**.

☐ Run **STEP 0**. Wait for ✅ `Ready!`

☐ Run the ▶️ data cell and the **three** ▶️ **tool cells** under it (baselines, features, backtest functions). Everything below needs them.

☐ Run every cell in order. Each ✍️ cell depends on the cells above it.

What a backtest is, in one table

Word	What it means today
Origin	The week you are "standing in" when you make the forecast. You may only use weeks **before** it
Horizon	How many weeks ahead you forecast: 1 (next week), 4 (the month-ahead roster), 8
Fold	One exam: train on the past, forecast a block of weeks, score it
Walk forward	Refit at each cut, forecast the next 4 weeks, score, slide 4 weeks on. The training block only grows
Rolling origin	Stand at **every** week, forecast `h` weeks blind, score, move one week on
One-step	Each week is forecast from last week's **real** count (the regression and one-step naive)
Blind	The whole block is forecast at the origin; no peeking at any week inside it

The exercise, step by step

Task	What you do	The tool you need
A1	The flat training mean on the last 8 weeks	`flat = train.mean()`
A2	The same method from every origin	`H_AHEAD = 8`

Task	What you do	The tool you need
B1	Walk the regression forward: six folds	`STEP = 4`, `H_FOLD = 4`
B2	The mean, and when the worst fold starts	`fold_mae.mean()`, `folds["regression"].idxmax()`
C1	One-step naive on the same folds	`NAIVE_COLUMN = "tickets_lag1"`
C2	Count the folds naive wins	`int((folds["naive"] < folds["regression"]).sum())`
C3	The rate forecast, blind for the whole fold	`RATE_H = 4`
D1	Every origin, three horizons	`HORIZONS = [1, 4, 8]`
D2	The best method 1 week and 8 weeks ahead	`summary.loc[1].idxmin()`, `summary.loc[8].idxmin()`
E1	The month-ahead roster pick	1, 2, 3 or 4
🌶️ **Bonus**	Refit every week: does the verdict survive?	`STEP_EVERY = 1`

The code you'll use

```
walk_forward_mae(tickets, make_features, X_COLS, step=4, h=4)   # six folds, one MAE
each

forecasters = {"naive": lambda past, h: naive(past["tickets_raised"], h)}
rolling_origin(tickets, forecasters, h=8)    # one row per origin, one column per method

summary.loc[1]            # the 1-week-ahead row of the summary table
summary.loc[1].idxmin()   # the NAME of the column with the smallest number
(a < b).sum()             # how many times a is smaller than b
```

`lambda past, h: ...` is a one-line function. `rolling_origin` calls it at every origin with `past` = only the weeks before that origin.

When things go wrong: error → fix

You see	It means	Fix
`AssertionError: ✍️ Fill in … first`	A `___` is still there	Replace it, run the cell again

You see	It means	Fix
`AssertionError: ✍️ Finish … first`	You skipped an earlier task	Do that task, then come back
`NameError: name 'walk_forward_mae' …` / `'rolling_origin'` / `'make_features'` / `'X_COLS'`	You skipped a ▶️ tool cell (or restarted)	Run the three ▶️ tool cells under STEP 0
`NameError: name 'tickets_lag1' is not defined`	You typed a column name without quotes	Column names are text: `"tickets_lag1"`
`ValueError: operands could not be broadcast together …` (C3)	`RATE_H` is longer than the 4-week fold	The fold is 4 weeks: `RATE_H = 4`
❌ in B2 with a date that looks right	You typed the date as words, e.g. `"20 Oct"`	Let pandas find it: `folds["regression"].idxmax()`
❌ in D2	You typed the name with a typo or without quotes	Use `summary.loc[1].idxmin()`: it returns the exact name
D1 takes a few seconds	It runs about 250 small forecasts per horizon	Normal. Wait for the table
Everything is weird	–	**Runtime → Restart session and run all**

Rules of thumb you'll keep forever

1. **One split is one screenshot.** Never trust a single held-out score on its own.
2. **Only the past at each origin, and refit every time.** That's what makes it a backtest.
3. **Same folds, same scorer, every method**, baselines included.
4. **Report the spread** across folds, and **when** the bad folds happen.
5. **Test at the horizon the user needs.** A forecast that needs last week's count can't make a month-ahead roster.
6. **Fix the backtest design before you look** (folds, step, horizon). Never tune on the folds you report.
7. **If the simple method wins or ties, recommend it**, in four plain sentences.

🏁 **Finished?** Run `scoreboard()`. Then try the 🌶️ task: refit every week and see whether the verdict survives.